

Министерство науки и высшего образования Российской  
Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИТМО

## ОТЧЕТ

по Лабораторной работе № 5

«Процедуры, функции, триггеры в PostgreSQL»

по дисциплине «Проектирование и реализация баз  
данных»

Обучающийся: Петрова Мария Валерьевна

Факультет: прикладной информатики

Группа: К3239

Направление подготовки: 09.03.03 Прикладная  
информатика

Образовательная программа: Мобильные и сетевые  
технологии 2023

Преподаватели: Говорова Марина Михайловна  
Белов Александр

Санкт-Петербург

2025 г.

# 1 Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

## 2 Практическое задание

В рамках лабораторной работы требовалось:

1. Создать три процедуры для индивидуальной базы данных согласно варианту.
2. Создать оригинальные триггеры для индивидуальной базы данных.
3. Выполнить тестирование разработанных объектов в среде SQL Shell (psql).
4. Подготовить подтверждающие скриншоты кода и результатов работы процедур и триггеров.

## 3 Выполнение работы

### 3.1 Наименование базы данных и вариант

Вариант: 1 — БД «ОТЕЛЬ».

Наименование базы данных: **hotel\_db**.

Схема базы данных: **hotel**.

## 3.2 Разработка процедур

Процедура 1. Увеличение цены всех номеров на 5%, если в отеле нет свободных номеров

```
hotel_db=# create or replace procedure hotel.raise_prices_if_no_free_rooms(  
hotel_db(#      in p_hotel_name varchar(60)  
hotel_db(# )  
hotel_db=# language plpgsql  
hotel_db=# as  
hotel_db=# $$  
hotel_db$# declare  
hotel_db$#     v_hotel_id bigint;  
hotel_db$#     v_free_rooms_count integer;  
hotel_db$# begin  
hotel_db$#     select h.id  
hotel_db$#     into v_hotel_id  
hotel_db$#     from hotel.hotel h  
hotel_db$#     where h.name = p_hotel_name;  
hotel_db$#  
hotel_db$#     if v_hotel_id is null then  
hotel_db$#         raise exception 'Отель с названием "%" не найден', p_hotel_name;  
hotel_db$#     end if;  
hotel_db$#  
hotel_db$#     select count(*)  
hotel_db$#     into v_free_rooms_count  
hotel_db$#     from hotel.room r  
hotel_db$#     where r.hotel_id = v_hotel_id  
hotel_db$#           and r.occupancy_status = 'свободен';  
hotel_db$#  
hotel_db$#     if v_free_rooms_count = 0 then  
hotel_db$#         update hotel.price p  
hotel_db$#         set price_value = ceil(p.price_value * 1.05)  
hotel_db$#         where p.room_type_id in (  
hotel_db$#             select distinct r.room_type_id  
hotel_db$#             from hotel.room r  
hotel_db$#             where r.hotel_id = v_hotel_id  
hotel_db$#         );  
hotel_db$#  
hotel_db$#         raise notice 'Цены для отеля "%" увеличены на 5%%', p_hotel_name;  
hotel_db$#     else  
hotel_db$#         raise notice 'В отеле "%" есть свободные номера (%), цены не изменены',  
hotel_db$#             p_hotel_name, v_free_rooms_count;  
hotel_db$#     end if;  
hotel_db$# end;  
hotel_db$# $$;  
CREATE PROCEDURE
```

Рис. 1: Код процедуры 1

```

hotel_db=# select
hotel_db=#     h.name as hotel_name,
hotel_db=#     rt.name as room_type,
hotel_db=#     p.price_value
hotel_db=# from hotel.hotel h
hotel_db=# join hotel.room r on r.hotel_id = h.id
hotel_db=# join hotel.room_type rt on rt.id = r.room_type_id
hotel_db=# join hotel.price p on p.room_type_id = rt.id
hotel_db=# where h.name = 'Отель Аврора'
hotel_db=# order by rt.name, p.price_value;
  hotel_name | room_type | price_value
-----+-----+-----
  Отель Аврора | двухместный |          5250
  Отель Аврора | двухместный |          6200
  Отель Аврора | люкс       |         15750
  Отель Аврора | люкс       |         18500
  Отель Аврора | одноместный |          3675
  Отель Аврора | одноместный |          4200
(6 rows)

```

Рис. 2: Проверка данных до вызова процедуры 1

```

CALL
hotel_db=# call hotel.raise_prices_if_no_free_rooms('Отель Аврора');
NOTICE:  Цены для отеля "Отель Аврора" увеличены на 5%
CALL

```

Рис. 3: Вызов процедуры 1

```

CALL
hotel_db=# select
hotel_db=#     h.name as hotel_name,
hotel_db=#     rt.name as room_type,
hotel_db=#     p.price_value
hotel_db=# from hotel.hotel h
hotel_db=# join hotel.room r on r.hotel_id = h.id
hotel_db=# join hotel.room_type rt on rt.id = r.room_type_id
hotel_db=# join hotel.price p on p.room_type_id = rt.id
hotel_db=# where h.name = 'Отель Аврора'
hotel_db=# order by rt.name, p.price_value;
  hotel_name | room_type | price_value
-----+-----+-----
  Отель Аврора | двухместный |         5513
  Отель Аврора | двухместный |         6510
  Отель Аврора | люкс       |        16538
  Отель Аврора | люкс       |        19425
  Отель Аврора | одноместный |         3859
  Отель Аврора | одноместный |         4410
(6 rows)

```

Рис. 4: Проверка данных после вызова процедуры 1

Процедура `raise_prices_if_no_free_rooms` принимает название отеля, проверяет наличие свободных номеров и при их отсутствии увеличивает цены на 5% для типов номеров, используемых в данном отеле. Проверила,



что при наличии свободных номеров цены не изменяются, а при отсутствии свободных мест стоимость автоматически пересчитывается.

## Процедура 2. Получение информации о свободных одноместных номерах отеля на завтрашний день

```
hotel_db=# create or replace procedure hotel.get_free_single_rooms_tomorrow(
hotel_db(#   in p_hotel_name varchar(60),
hotel_db(#   inout p_result refcursor
hotel_db(# )
hotel_db=# language plpgsql
hotel_db=# as
hotel_db=# $$
hotel_db$$ begin
hotel_db$$   open p_result for
hotel_db$$       select
hotel_db$$           h.name as hotel_name,
hotel_db$$           h.city,
hotel_db$$           r.room_number,
hotel_db$$           rt.name as room_type,
hotel_db$$           r.occupancy_status
hotel_db$$       from hotel.room r
hotel_db$$       join hotel.hotel h on h.id = r.hotel_id
hotel_db$$       join hotel.room_type rt on rt.id = r.room_type_id
hotel_db$$       where h.name = p_hotel_name
hotel_db$$           and rt.name = 'одноместный'
hotel_db$$           and r.occupancy_status = 'свободен'
hotel_db$$       order by r.room_number;
hotel_db$$ end;
hotel_db=# $$;
CREATE PROCEDURE
hotel_db=#
```

Рис. 5: Код процедуры 2

```
hotel_db=# begin;
BEGIN
hotel_db=# call hotel.get_free_single_rooms_tomorrow('Отель Волга', 'cur_rooms');
p_result
-----
cur_rooms
(1 row)

hotel_db=# fetch all from cur_rooms;
 hotel_name | city | room_number | room_type | occupancy_status
-----+-----+-----+-----+-----
  Отель Волга | Казань |      108 | одноместный | свободен
(1 row)

hotel_db=# commit;
COMMIT
hotel_db=#
```

Рис. 6: Вызов процедуры 2 и результат её выполнения

Процедура `get_free_single_rooms_tomorrow` реализована с использованием параметра `INOUT refcursor`. Она позволяет получить список свободных одноместных номеров выбранного отеля. В ходе проверки процедура корректно открывала курсор и возвращала данные при выполнении команд `CALL` и `FETCH ALL` в среде `psql`.

### Процедура 3. Бронирование двухместного номера в гостинице на заданную дату и количество дней проживания

```
hotel_db=# create or replace procedure hotel.book_double_room_simple(
hotel_db(#      in p_hotel_name varchar(60),
hotel_db(#      in p_check_in_date date,
hotel_db(#      in p_days_count integer,
hotel_db(#      in p_guest_id bigint,
hotel_db(#      in p_contract_number varchar(30)
hotel_db(# )
hotel_db=# language plpgsql
hotel_db=# as
hotel_db=# $$
hotel_db=# declare
hotel_db=#     v_room_id bigint;
hotel_db=#     v_room_type_id bigint;
hotel_db=#     v_price integer;
hotel_db=# begin
hotel_db=#     select
hotel_db=#         r.id,
hotel_db=#         r.room_type_id
hotel_db=#     into
hotel_db=#         v_room_id,
hotel_db=#         v_room_type_id
hotel_db=#     from hotel.room r
hotel_db=#     join hotel.hotel h on h.id = r.hotel_id
hotel_db=#     join hotel.room_type rt on rt.id = r.room_type_id
hotel_db=#     where h.name = p_hotel_name
hotel_db=#           and rt.name = 'двухместный'
hotel_db=#           and r.occupancy_status = 'свободен'
hotel_db=#     order by r.room_number
hotel_db=#     limit 1;
hotel_db=#
hotel_db=#     if v_room_id is null then
hotel_db=#         raise exception 'Свободный двухместный номер в отеле "%" не найден', p_hotel_name;
hotel_db=#     end if;
hotel_db=#
hotel_db=#     select p.price_value
hotel_db=#     into v_price
hotel_db=#     from hotel.price p
hotel_db=#     where p.room_type_id = v_room_type_id
hotel_db=#     limit 1;
hotel_db=#
hotel_db=#     insert into hotel.booking (
hotel_db=#         room_id,
hotel_db=#         guest_id,
hotel_db=#         contract_number,
hotel_db=#         booking_date,
hotel_db=#         check_in_date,
hotel_db=#         check_out_date,
hotel_db=#         payment_date,
hotel_db=#         payment_type,
hotel_db=#         payment_method,
hotel_db=#         status,
hotel_db=#         total_cost,
hotel_db=#         comment
hotel_db=#     )
hotel_db=#     values (
hotel_db=#         v_room_id,
hotel_db=#         p_guest_id,
hotel_db=#         p_contract_number,
hotel_db=#         current_date,
hotel_db=#         p_check_in_date,
hotel_db=#         p_check_in_date + p_days_count,
hotel_db=#         null,
hotel_db=#         'полная',
hotel_db=#         'онлайн',
hotel_db=#         'забронирован',
hotel_db=#         v_price * p_days_count,
hotel_db=#         'Бронирование создано процедурой'
hotel_db=#     );
hotel_db=#
hotel_db=#     update hotel.room
hotel_db=#     set occupancy_status = 'забронирован'
hotel_db=#     where id = v_room_id;
hotel_db=#
hotel_db=#     raise notice 'Бронирование успешно создано. Номер id = %', v_room_id;
hotel_db=# end;
hotel_db=# $$;
CREATE PROCEDURE
```

Рис. 7: Код процедуры 3

```

hotel_db=# select
hotel_db=#     h.name as hotel_name,
hotel_db=#     r.id as room_id,
hotel_db=#     r.room_number,
hotel_db=#     r.occupancy_status
hotel_db=# from hotel.room r
hotel_db=# join hotel.hotel h on h.id = r.hotel_id
hotel_db=# join hotel.room_type rt on rt.id = r.room_type_id
hotel_db=# where h.name = 'Отель Невский'
hotel_db=# and rt.name = 'двухместный'
hotel_db=# order by r.room_number;

```

| hotel_name    | room_id | room_number | occupancy_status |
|---------------|---------|-------------|------------------|
| Отель Невский | 2       | 102         | забронирован     |
| Отель Невский | 21      | 202         | свободен         |

(2 rows)

Рис. 8: Проверка данных до вызова процедуры 3

```

hotel_db=# call hotel.book_double_room_simple(
hotel_db(#     'Отель Невский',
hotel_db(#     '2026-04-25',
hotel_db(#     3,
hotel_db(#     1,
hotel_db(#     'DOG-001'
[hotel_db(# );
NOTICE: Бронирование успешно создано. Номер id = 2
CALL
hotel_db=# █

```

Рис. 9: Вызов процедуры 3

```

hotel_db=# select
hotel_db=#     h.name as hotel_name,
hotel_db=#     r.id as room_id,
hotel_db=#     r.room_number,
hotel_db=#     r.occupancy_status
hotel_db=# from hotel.room r
hotel_db=# join hotel.hotel h on h.id = r.hotel_id
hotel_db=# join hotel.room_type rt on rt.id = r.room_type_id
hotel_db=# where h.name = 'Отель Невский'
hotel_db=# and rt.name = 'двухместный'
hotel_db=# order by r.room_number;

```

| hotel_name    | room_id | room_number | occupancy_status |
|---------------|---------|-------------|------------------|
| Отель Невский | 2       | 102         | забронирован     |
| Отель Невский | 21      | 202         | забронирован     |

(2 rows)

hotel\_db=# █

Рис. 10: Проверка данных после вызова процедуры 3

Процедура `book_double_room_simple` предназначена для бронирования двухместного номера в выбранном отеле. В качестве параметров используются название отеля, дата заезда, количество дней проживания, идентификатор гостя и номер договора. Процедура находила первый свободный двухместный номер, добавляла запись в таблицу `booking` и изменяла статус номера на `забронирован`.

### 3.3 Разработка триггеров

**Триггер 1. Автоматическая установка статуса номера «забронирован» после создания бронирования**

```
hotel_db=# create or replace function hotel.fn_set_room_booked()
hotel_db=# returns trigger
hotel_db=# as
hotel_db=# $$
hotel_db$# begin
hotel_db$#     update hotel.room
hotel_db$#     set occupancy_status = 'забронирован'
hotel_db$#     where id = new.room_id;
hotel_db$#
hotel_db$#     return new;
hotel_db$# end;
hotel_db$# $$
hotel_db=# language plpgsql;
CREATE FUNCTION
hotel_db=#
```

Рис. 11: Код триггерной функции для триггера 1

```
hotel_db=# create trigger trg_set_room_booked
hotel_db=#
hotel_db=# after insert on hotel.booking
hotel_db=#
hotel_db=# for each row
hotel_db=#
hotel_db=# execute function hotel.fn_set_room_booked();
CREATE TRIGGER
hotel_db=#
```

Рис. 12: Создание триггера 1

```

hotel_db=# select
hotel_db=#     r.id,
hotel_db=#     r.room_number,
hotel_db=#     r.occupancy_status
hotel_db=# from hotel.room r
hotel_db=# where r.occupancy_status = 'свободен'
hotel_db=# order by r.id
hotel_db=# limit 5;
 id | room_number | occupancy_status
-----+-----+-----
  3 |         201 | свободен
  4 |         103 | свободен
  5 |         104 | свободен
  6 |         301 | свободен
  8 |         205 | свободен
(5 rows)

hotel_db=#

```

Рис. 13: Проверка данных до срабатывания триггера 1



```

hotel_db=# insert into hotel.booking (
hotel_db(#      room_id,
hotel_db(#      guest_id,
hotel_db(#      contract_number,
hotel_db(#      booking_date,
hotel_db(#      check_in_date,
hotel_db(#      check_out_date,
hotel_db(#      payment_date,
hotel_db(#      payment_type,
hotel_db(#      payment_method,
hotel_db(#      status,
hotel_db(#      total_cost
hotel_db(# )
hotel_db=# values (
hotel_db(#      3,
hotel_db(#      1,
hotel_db(#      'DOG-001',
hotel_db(#      current_date,
hotel_db(#      '2026-05-01',
hotel_db(#      '2026-05-03',
hotel_db(#      null,
hotel_db(#      'полная',
hotel_db(#      'онлайн',
hotel_db(#      'забронирован',
hotel_db(#      12400
hotel_db(# );
INSERT 0 1

```

Рис. 14: Вызов действия, запускающего триггер 1

```

hotel_db=# select
hotel_db-#      id,
hotel_db-#      room_number,
hotel_db-#      occupancy_status
hotel_db-# from hotel.room
hotel_db-# where id = 3;
  id | room_number | occupancy_status
-----+-----+-----
   3 |         201 | забронирован
(1 row)

```

Рис. 15: Проверка данных после срабатывания триггера 1

Триггер 1 был реализован для автоматической синхронизации таблиц `booking` и `room`. После вставки нового бронирования он автоматически переводит соответствующий номер в состояние **забронирован**.

**Триггер 2. Автоматическая установка статуса номера «занят» при заселении гостя**

```
hotel_db=# create or replace function hotel.fn_set_room_occupied()
hotel_db=# returns trigger
hotel_db=# as
hotel_db=# $$
hotel_db$# begin
hotel_db$#     if new.status = 'заселен' then
hotel_db$#         update hotel.room
hotel_db$#             set occupancy_status = 'занят'
hotel_db$#             where id = new.room_id;
hotel_db$#     end if;
hotel_db$#
hotel_db$#     return new;
hotel_db$# end;
hotel_db$# $$
hotel_db=# language plpgsql;
CREATE FUNCTION
hotel_db=#
```

Рис. 16: Код триггерной функции для триггера 2

```
hotel_db=# create trigger trg_set_room_occupied
hotel_db=# after update of status on hotel.booking
hotel_db=# for each row
hotel_db=# execute function hotel.fn_set_room_occupied();
CREATE TRIGGER
hotel_db=#
```

Рис. 17: Создание триггера 2

```
hotel_db=# select
hotel_db=#     b.id as booking_id,
hotel_db=#     b.room_id,
hotel_db=#     b.status as booking_status,
hotel_db=#     r.occupancy_status as room_status
hotel_db=# from hotel.booking b
hotel_db=# join hotel.room r on r.id = b.room_id
hotel_db=# where b.id = 18;
 booking_id | room_id | booking_status | room_status
-----+-----+-----+-----
          18 |         3 | забронирован   | забронирован
(1 row)
```

Рис. 18: Проверка данных до срабатывания триггера 2

```

hotel_db=# update hotel.booking
hotel_db=# set status = 'заселен'
hotel_db=# where id = 18;

```

Рис. 19: Вызов действия, запускающего триггер 2

```

hotel_db=# select
hotel_db=#     b.id as booking_id,
hotel_db=#     b.room_id,
hotel_db=#     b.status as booking_status,
hotel_db=#     r.occupancy_status as room_status
hotel_db=# from hotel.booking b
hotel_db=# join hotel.room r on r.id = b.room_id
hotel_db=# where b.id = 18;
 booking_id | room_id | booking_status | room_status
-----+-----+-----+-----
          18 |        3 |   заселен     |   занят
(1 row)

```

Рис. 20: Проверка данных после срабатывания триггера 2

Триггер 2 реализует изменение статуса номера на **занят** в тот момент, когда статус бронирования изменяется на **заселен**. Это позволяет автоматически поддерживать корректное состояние номера.

**Триггер 3. Автоматическая установка статуса номера «свободен» при завершении или отмене бронирования**

```

hotel_db=# create or replace function hotel.fn_set_room_free()
hotel_db=# returns trigger
hotel_db=# as
hotel_db=# $$
hotel_db$# begin
hotel_db$#     if new.status in ('завершен', 'отменен') then
hotel_db$#         update hotel.room
hotel_db$#         set occupancy_status = 'свободен'
hotel_db$#         where id = new.room_id;
hotel_db$#     end if;
hotel_db$#     return new;
hotel_db$# end;
hotel_db$# $$
hotel_db=# language plpgsql;
CREATE FUNCTION
hotel_db=#

```

Рис. 21: Код триггерной функции для триггера 3



```

hotel_db=# create trigger trg_set_room_free
hotel_db=# after update of status on hotel.booking
hotel_db=# for each row
hotel_db=# execute function hotel.fn_set_room_free();
CREATE TRIGGER
hotel_db=#

```

Рис. 22: Создание триггера 3

```

hotel_db=# select
hotel_db=#     b.id as booking_id,
hotel_db=#     b.room_id,
hotel_db=#     b.status as booking_status,
hotel_db=#     r.occupancy_status as room_status
hotel_db=# from hotel.booking b
hotel_db=# join hotel.room r on r.id = b.room_id
hotel_db=# where b.id = 18;
 booking_id | room_id | booking_status | room_status
-----+-----+-----+-----
          18 |      3 |   заселен     |   занят
(1 row)

```

Рис. 23: Проверка данных до срабатывания триггера 3

```

hotel_db=# update hotel.booking
hotel_db=# set status = 'завершен'
hotel_db=# where id = 18;
UPDATE 1

```

Рис. 24: Вызов действия, запускающего триггер 3

```

hotel_db=# select
hotel_db=#     b.id as booking_id,
hotel_db=#     b.room_id,
hotel_db=#     b.status as booking_status,
hotel_db=#     r.occupancy_status as room_status
hotel_db=# from hotel.booking b
hotel_db=# join hotel.room r on r.id = b.room_id
hotel_db=# where b.id = 18;
 booking_id | room_id | booking_status | room_status
-----+-----+-----+-----
          18 |      3 |   завершен     |   свободен
(1 row)

```

Рис. 25: Проверка данных после срабатывания триггера 3

Триггер 3 обеспечивает перевод номера в состояние **свободен**, если бронирование получает статус **завершен** или **отменен**. Поддерживается согласованность статусов между таблицами **booking** и **room**.

#### Триггер 4. Автоматический пересчёт общей стоимости бронирования

```
hotel_db=# create or replace function hotel.fn_calculate_booking_total_cost()
hotel_db=# returns trigger
hotel_db=# as
hotel_db=# $$
hotel_db$# declare
hotel_db$#     v_room_type_id bigint;
hotel_db$#     v_price integer;
hotel_db$#     v_days integer;
hotel_db$# begin
hotel_db$#     select r.room_type_id
hotel_db$#     into v_room_type_id
hotel_db$#     from hotel.room r
hotel_db$#     where r.id = new.room_id;
hotel_db$#
hotel_db$#     select p.price_value
hotel_db$#     into v_price
hotel_db$#     from hotel.price p
hotel_db$#     where p.room_type_id = v_room_type_id
hotel_db$#           and new.check_in_date between p.start_date and p.end_date
hotel_db$#     order by p.id desc
hotel_db$#     limit 1;
hotel_db$#
hotel_db$#     v_days := new.check_out_date - new.check_in_date;
hotel_db$#     new.total_cost := v_price * v_days;
hotel_db$#     return new;
hotel_db$# end;
hotel_db$# $$
hotel_db=# language plpgsql;
CREATE FUNCTION
hotel_db=#
```

Рис. 26: Код триггерной функции для триггера 4

```
hotel_db=# create trigger trg_calculate_booking_total_cost
hotel_db=# before insert or update of room_id, check_in_date, check_out_date
hotel_db=# on hotel.booking
hotel_db=# for each row
hotel_db=# execute function hotel.fn_calculate_booking_total_cost();
CREATE TRIGGER
hotel_db=#
```

Рис. 27: Создание триггера 4

```

hotel_db=# select
hotel_db=#     id,
hotel_db=#     room_id,
hotel_db=#     check_in_date,
hotel_db=#     check_out_date,
hotel_db=#     total_cost
hotel_db=# from hotel.booking
hotel_db=# where id = 18;
 id | room_id | check_in_date | check_out_date | total_cost
-----+-----+-----+-----+-----
 18 |      3 | 2026-05-01   | 2026-05-04   |      58275
(1 row)

```

Рис. 28: Проверка данных до срабатывания триггера 4

```

hotel_db=# update hotel.booking
hotel_db=# set check_out_date = check_out_date + 1
hotel_db=# where id = 18;
UPDATE 1
hotel_db=#

```

Рис. 29: Вызов действия, запускающего триггер 4

```

hotel_db=# update hotel.booking
hotel_db=# set check_out_date = check_out_date + 1
hotel_db=# where id = 18;
UPDATE 1
hotel_db=# select
hotel_db=#     id,
hotel_db=#     room_id,
hotel_db=#     check_in_date,
hotel_db=#     check_out_date,
hotel_db=#     total_cost
hotel_db=# from hotel.booking
hotel_db=# where id = 18;
 id | room_id | check_in_date | check_out_date | total_cost
-----+-----+-----+-----+-----
 18 |      3 | 2026-05-01   | 2026-05-05   |      77700
(1 row)

```

Рис. 30: Проверка данных после срабатывания триггера 4

Триггер 4 был разработан для автоматического пересчёта поля `total_cost` при изменении параметров проживания. Проверила, что при изменении даты выезда итоговая стоимость корректно пересчитывается на основании цены номера и продолжительности проживания.

## Триггер 5. Синхронизация статуса уборки номера со статусом записи в таблице cleaning

```
hotel_db=# create or replace function hotel.fn_sync_room_cleaning_status()
hotel_db=# returns trigger
hotel_db=# as
hotel_db=# $$
hotel_db$$ begin
hotel_db$$     update hotel.room
hotel_db$$         set cleaning_status = new.status
hotel_db$$         where id = new.room_id;
hotel_db$$
hotel_db$$     return new;
hotel_db$$ end;
hotel_db$$ $$
[hotel_db=# language plpgsql;
CREATE FUNCTION
```

Рис. 31: Код триггерной функции для триггера 5

```
hotel_db=# create trigger trg_sync_room_cleaning_status
hotel_db=# after insert or update of status on hotel.cleaning
hotel_db=# for each row
hotel_db=# execute function hotel.fn_sync_room_cleaning_status();
CREATE TRIGGER
hotel_db=#
```

Рис. 32: Создание триггера 5

```
hotel_db=# select
hotel_db=#     c.id as cleaning_id,
hotel_db=#     c.room_id,
hotel_db=#     c.status as cleaning_status,
hotel_db=#     r.cleaning_status as room_cleaning_status
hotel_db=# from hotel.cleaning c
hotel_db=# join hotel.room r on r.id = c.room_id
hotel_db=# where c.id = 3;
 cleaning_id | room_id | cleaning_status | room_cleaning_status
-----+-----+-----+-----
          3 |      3 |    убран        |    убран
(1 row)
```

Рис. 33: Проверка данных до срабатывания триггера 5

```
hotel_db=# update hotel.cleaning
hotel_db=# set status = 'не убран'
hotel_db=# where id = 3;
UPDATE 1
```

Рис. 34: Вызов действия, запускающего триггер 5



```

hotel_db=# select
hotel_db=#     c.id as cleaning_id,
hotel_db=#     c.room_id,
hotel_db=#     c.status as cleaning_status,
hotel_db=#     r.cleaning_status as room_cleaning_status
hotel_db=# from hotel.cleaning c
hotel_db=# join hotel.room r on r.id = c.room_id
hotel_db=# where c.id = 3;
  cleaning_id | room_id | cleaning_status | room_cleaning_status
-----+-----+-----+-----
          3 |      3 |   не убран     |   не убран
(1 row)

```

Рис. 35: Проверка данных после срабатывания триггера 5

Триггер 5 обеспечивает синхронизацию данных между таблицами `cleaning` и `room`. При изменении статуса записи об уборке триггер автоматически обновляет поле `cleaning_status` соответствующего номера.

### Триггер 6. Автоматическая установка статуса уборки «не убран» после завершения бронирования

```

hotel_db=# create or replace function hotel.fn_set_room_not_clean_after_checkout()
hotel_db=# returns trigger
hotel_db=# as
hotel_db=# $$
hotel_db$# begin
hotel_db$#     if new.status = 'завершен' then
hotel_db$#         update hotel.room
hotel_db$#             set cleaning_status = 'не убран'
hotel_db$#             where id = new.room_id;
hotel_db$#     end if;
hotel_db$#     return new;
hotel_db$# end;
hotel_db$# $$
hotel_db=# language plpgsql;
CREATE FUNCTION

```

Рис. 36: Код триггерной функции для триггера 6

```

hotel_db=# create trigger trg_set_room_not_clean_after_checkout
hotel_db=# after update of status on hotel.booking
hotel_db=# for each row
hotel_db=# execute function hotel.fn_set_room_not_clean_after_checkout();
CREATE TRIGGER

```

Рис. 37: Создание триггера 6



```

hotel_db=# select
hotel_db-#      b.id as booking_id,
hotel_db-#      b.room_id,
hotel_db-#      b.status as booking_status,
hotel_db-#      r.cleaning_status as room_cleaning_status
hotel_db-# from hotel.booking b
hotel_db-# join hotel.room r on r.id = b.room_id
[hotel_db-# where b.id = 5;
  booking_id | room_id | booking_status | room_cleaning_status
-----+-----+-----+-----
          5 |      8 |   заселен     |   убран
(1 row)

```

Рис. 38: Проверка данных до срабатывания триггера 6

```

hotel_db=# update hotel.booking
hotel_db-# set status = 'завершен'
hotel_db-# where id = 5;
UPDATE 1

```

Рис. 39: Вызов действия, запускающего триггер 6

```

hotel_db=# select
hotel_db-#      b.id as booking_id,
hotel_db-#      b.room_id,
hotel_db-#      b.status as booking_status,
hotel_db-#      r.cleaning_status as room_cleaning_status
hotel_db-# from hotel.booking b
hotel_db-# join hotel.room r on r.id = b.room_id
[hotel_db-# where b.id = 5;
  booking_id | room_id | booking_status | room_cleaning_status
-----+-----+-----+-----
          5 |      8 |   завершен     |   не убран
(1 row)

```

Рис. 40: Проверка данных после срабатывания триггера 6

Триггер 6 предназначен для установки статуса **не убран** в поле `cleaning_status` таблицы `room` после завершения бронирования. Это соответствует логике предметной области, поскольку после выезда гостя номер должен быть подготовлен к уборке.

## Триггер 7. Запрет создания бронирования для номера, который не является свободным

```
hotel_db=# create or replace function hotel.fn_check_room_is_free_before_booking()
hotel_db=# returns trigger
hotel_db=# as
hotel_db=# $$
hotel_db=# declare
hotel_db=#     v_room_status varchar(30);
hotel_db=# begin
hotel_db=#     select r.occupancy_status
hotel_db=#     into v_room_status
hotel_db=#     from hotel.room r
hotel_db=#     where r.id = new.room_id;
hotel_db=#
hotel_db=#     if v_room_status is distinct from 'свободен' then
hotel_db=#         raise exception 'Невозможно создать бронирование: номер с id = % не свободен', new.room_id;
hotel_db=#     end if;
hotel_db=#
hotel_db=#     return new;
hotel_db=# end;
hotel_db=# $$
hotel_db=# language plpgsql;
CREATE FUNCTION
```

Рис. 41: Код триггерной функции для триггера 7

```
hotel_db=# create trigger trg_check_room_is_free_before_booking
hotel_db=# before insert on hotel.booking
hotel_db=# for each row
hotel_db=# execute function hotel.fn_check_room_is_free_before_booking();
CREATE TRIGGER
hotel_db=#
```

Рис. 42: Создание триггера 7

```
hotel_db=# select
hotel_db=#     id,
hotel_db=#     room_number,
hotel_db=#     occupancy_status
hotel_db=# from hotel.room
hotel_db=# where occupancy_status <> 'свободен'
hotel_db=# order by id
hotel_db=# limit 10;
```

| id | room_number | occupancy_status |
|----|-------------|------------------|
| 1  | 101         | занят            |
| 2  | 102         | забронирован     |
| 7  | 401         | занят            |
| 9  | 106         | забронирован     |
| 12 | 107         | занят            |
| 14 | 109         | забронирован     |
| 16 | 111         | занят            |
| 18 | 101         | занят            |
| 19 | 102         | занят            |
| 20 | 201         | занят            |

(10 rows)

Рис. 43: Проверка данных до срабатывания триггера 7

```

hotel_db=# insert into hotel.booking (
hotel_db(#   room_id,
hotel_db(#   guest_id,
hotel_db(#   contract_number,
hotel_db(#   booking_date,
hotel_db(#   check_in_date,
hotel_db(#   check_out_date,
hotel_db(#   payment_date,
hotel_db(#   payment_type,
hotel_db(#   payment_method,
hotel_db(#   status,
hotel_db(#   total_cost,
hotel_db(#   comment
hotel_db(# )
hotel_db=# values (
hotel_db(#   2,
hotel_db(#   1,
hotel_db(#   'DOG-001',
hotel_db(#   current_date,
hotel_db(#   '2026-05-10',
hotel_db(#   '2026-05-12',
hotel_db(#   null,
hotel_db(#   'полная',
hotel_db(#   'онлайн',
hotel_db(#   'забронирован',
hotel_db(#   10000,
hotel_db(#   'Проверка 7 триггера'
hotel_db(# );
ERROR:  Невозможно создать бронирование: номер с id = 2 не свободен
CONTEXT:  PL/pgSQL function hotel.fn_check_room_is_free_before_booking() line 11 at RAISE
hotel_db=# █

```

Рис. 44: Вызов действия, запускающего триггер 7

Триггер 7 был реализован для контроля целостности данных. Он не позволяет вставить новую запись в таблицу **booking**, если выбранный номер уже имеет статус, отличный от **свободен**. При тестировании попытка создать бронирование для несвободного номера завершалась ошибкой.

## 4 Выводы

В ходе выполнения лабораторной работы были разработаны и протестированы три процедуры и семь триггеров для базы данных PostgreSQL предметной области «Отель». Процедуры обеспечили автоматизацию повышения цен, поиска свободных номеров и оформления бронирования. Разработанные триггеры позволили поддерживать согласованность данных между связанными таблицами, автоматически изменять статусы номеров и уборки, пересчитывать стоимость бронирования, а также контролировать корректность вставки новых записей. Проверка в SQL Shell (psql) показала, что все созданные объекты работают корректно, а цель лабораторной работы была достигнута.